

使用	建構子constructor	函式
是否使用型別	不需要	需要
	explicit 類別名 (型別宣告 參數1名稱){ }	void 函式名稱 (型別宣告 參數1名稱){ }
{ }	: 物件 (參數1)也就是「初始化列表」	物件=參數1 ;
Example	<pre>class EA{ public: explicit EA(int a) : b(a) { } private: int b; };</pre>	<pre>class EA{ public: void function1(int a){ b=a; } private: int b; };</pre>
呼叫時機	物件建立時，自動呼叫	在驅動程式手動呼叫
用途	初始化物件	修改或操作物件
是否會隱式轉換	可能發生	不會
是否需要顯示聲明	單個參數需要 無參數、多個參數不需要	不需要
呼叫方式	EA e(123);	EA e; A.function1(123);

Build Header		
.h	<pre> #include <iostream> class EA{ public: // 無參數 (預設) EA() :b(0),c(0){ } // 一個參數 explicit EA(int a) :b(a),c(0){ } // 兩個參數 EA(int a, int t) :b(a),c(t){ } private: int b,c; } </pre>	<pre> #include <iostream> class EA{ public: // 無參數 (預設) void function1() { b = 0; c = 0; } // 一個參數 void function2(int a) { b = a; c = 0; } // 兩個參數 void function3(int a, int t) { b = a; c = t; } private: int b,c; } </pre>
.cpp(driver program)	<pre> #include "EA.h" using namespace std; int main(){ EA e; EA e1(); EA e2(123); EA e3(123,246); } </pre>	<pre> #include "EA.h" using namespace std; int main(){ EA A; A.function1(); A.function2(123); A.function3(123,246); } </pre>

Information hiding & interface separation		
.h (interface)	<pre>#include <iostream> class EA{ public: // 無參數 (預設) EA() // 一個參數 explicit EA(int a) // 兩個參數 EA(int a, int t) private: int b,c; }</pre>	<pre>#include <iostream> class EA{ public: //無參數 (預設) void function1(; //一個參數 void function2(int a); //兩個參數 void function3(int a, int t); private: int b,c; }</pre>
.cpp (function prototypes)	<pre>#include "EA.h" using namespace std; EA::EA() :b(0),c(0){ } EA::EA(int a) :b(a),c(0){ } EA::EA(int a, int t) :b(a),c(t){ }</pre>	<pre>#include "EA.h" using namespace std; void EA::function1(){ b=0; c=0; } void EA::function2(int a){ b=a; c=0; } void EA::function3(int a, int t){ b=a; c=t; }</pre>
.cpp(driver program)	<pre>#include "EA.h" using namespace std; int main(){ EA e1(); EA e2(123); EA e3(123,246); }</pre>	<pre>#include "EA.h" using namespace std; int main(){ EA e; e.function1(); e.function2(123); e.function3(123,246); }</pre>